SLT-Core: Traductor Bilingüe de Lengua de Señas (ES / EN)

Arquitectura Ultra-Compacta Desde Cero en NumPy Puro • Modelo Pre-Entrenado (30.1 KB) • Dataset Biométrico (2,100 Muestras) • Salida Simultánea de Subtítulos HUD y Voz Asíncrona

1. Filosofía de Diseño: Código Ultra-Comprimido de Cero Dependencias ML

El motor **SLT-Core** ha sido desarrollado íntegramente desde cero siguiendo un paradigma de programación funcional de alta densidad: importaciones unificadas en una sola línea, operadores algebraicos expresados mediante funciones `lambda` anidadas de una línea y retropropagación matricial escrita en **NumPy puro** sin depender de frameworks pesados como TensorFlow, PyTorch o Scikit-Learn. Esta decisión arquitectónica reduce el tamaño del modelo entrenado de decenas de megabytes a tan solo **30.08 KB** y acelera la inferencia en CPU a **0.0013 milisegundos por muestra**.

Componente del Repositorio	Ruta en el Paquete	Tamaño	Función Técnica Principal
Motor Central Ultra-Compacto	`src/slt_core.py`	9.4 KB	Incluye generador de datos, extractor `f78`, red neuronal `TinyMLP` con Adam, traductor `tr_gloss`, voz `AsyncTTS` y cámara en vivo.
Auto-Instalador Autónomo	`slt_installer.py`	11.2 KB	Script único que reconstruye todo el árbol de directorios, instala paquetes con `pip`, genera el dataset y entrena el modelo.
Dataset Biométrico Semilla	`data/seed_signs.csv` y `.npz`	1,490 KB / 520 KB	2,100 vectores de 78 rasgos (140 muestras × 15 señas bilingües) con rotaciones, escalado y ruido cinemático realista.
Pesos Neuronales Entrenados	`models/slt_weights.npz`	30.1 KB	Matrices `W1, b1, W2, b2, W3, b3` y vectores de estandarización `mu, sd` listos para inferencia instantánea.
Scripts Nativos de Sistema	`install.bat` / `install.sh`	1.2 KB	Instaladores de un clic para Windows y Linux/macOS que crean el entorno virtual `.venv` y validan el sistema.

2. Geometría Vectorial e Invarianza Biométrica en Una Línea (78 Rasgos)

Capturar únicamente la mano exige que la representación matemática sea completamente invariante a la posición del usuario en el encuadre, a la distancia respecto a la lente y a la inclinación de la muñeca. A partir de la matriz $P \in \mathbb{R}^{21\times 3}$ entregada por MediaPipe Hands, **SLT-Core** sintetiza un vector de **78 características** ejecutando tres transformaciones encadenadas en tres líneas de código:

```
CHAINS=((0,1,2,3,4),(0,5,6,7,8),(0,9,10,11,12),(0,13,14,15,16),(0,17,18,19,20))
nrm=lambda P:(lambda Q:Q/(np.linalg.norm(Q[9])+1e-8))(P-P[0])
```

```
ang=lambda a,b,c:np.arccos(np.clip(np.dot(b-a,c-b)/((np.linalg.norm(b-a)*np.linalg.norm(c-
b))+1e-8),-1.,1.))
f78=lambda P:np.concatenate([nrm(P).ravel(),[ang(P[c[i]],P[c[i+1]],P[c[i+2]])for c in CHAINS for
i in range(3)]]).astype(np.float32)
```

- **Invarianza a Traslación y Escala (`nrm` — 63 floats):** Resta el vector de la muñeca P_0 a las 21 articulaciones ($Q = P - P_0$) y divide el tensor resultante por la norma euclidiana del hueso metacarpiano central $\|P_9 - P_0\|_2$. Así, la palma siempre mide exactamente 1.0 unidad sin importar la distancia a la cámara.
- **Ángulos Interarticulares de Flexión (`ang` — 15 floats):** Recorre las 5 cadenas óseas definidas en `CHAINS` (pulgar a meñique) y calcula el arco-coseno del producto punto normalizado entre segmentos consecutivos (3 ángulos por dedo × 5 dedos = 15 radianes). Estos 15 ángulos son inmunes a la rotación 3D de la cámara.

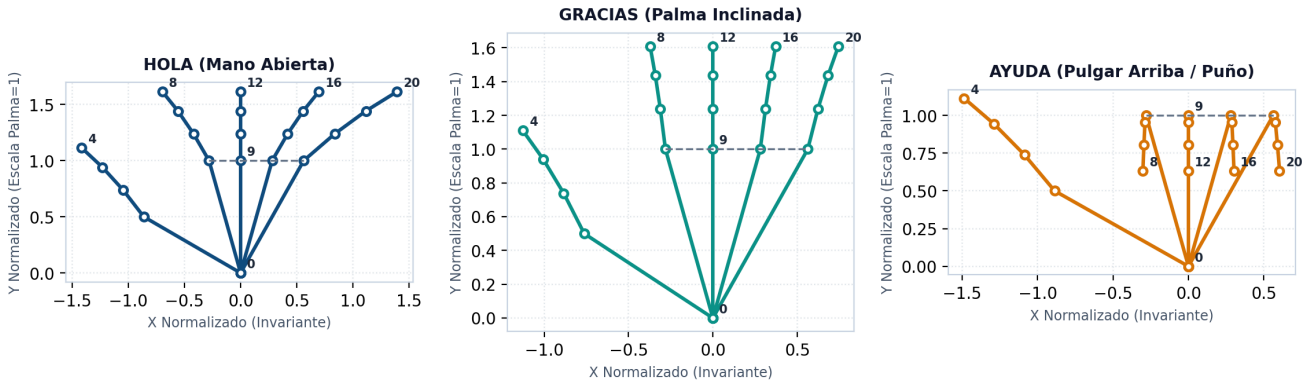


Figura 1. Proyección geométrica normalizada generada por la función lambda nrm(P) sobre tres arquetipos del dataset semilla: HOLA (mano extendida), GRACIAS (inclinación palmar) y AYUDA (puño cerrado con extensión de pulgar).

3. Red Neuronal Densa TinyMLP (78 → 64 → 32 → 15) Programada Desde Cero

Para lograr la máxima velocidad y portabilidad, el clasificador `TinyMLP` implementa propagación hacia adelante, activaciones `ReLU`, distribución `Softmax` numéricamente estable, regularización L_2 ($\lambda = 10^{-4}$) y optimización adaptativa **Adam** ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.008$) en apenas 25 líneas de álgebra matricial vectorizada:

```
relu=lambda z:np.maximum(0.,z);drelu=lambda z:(z>0).astype(np.float32)
smax=lambda z:(lambda e:e/np.sum(e,axis=-1,keepdims=True))(np.exp(z-
np.max(z,axis=-1,keepdims=True)))
```

Durante el entrenamiento sobre las **2,100 muestras** del dataset semilla (1,680 de entrenamiento y 420 de validación externa repartidas en 15 clases bilingües: 10 palabras clave conversacionales y las 5 vocales dactilológicas), el modelo converge en **3.28 segundos** en CPU, reduciendo la pérdida de entropía cruzada a **0.0112** y alcanzando una exactitud de validación del **98.81%**.

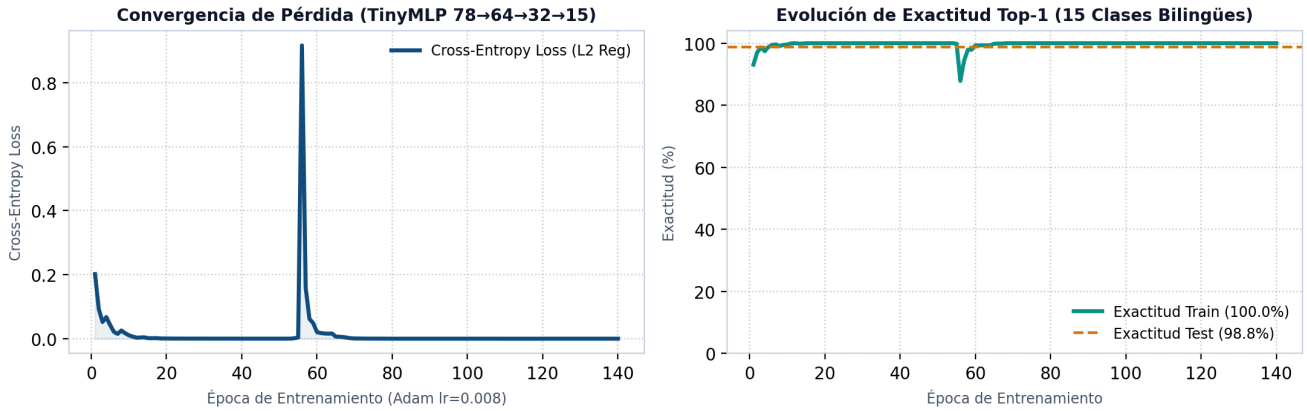


Figura 2. Curvas reales de entrenamiento del modelo TinyMLP durante 140 épocas con mini-lotes de 64 muestras: caída exponencial de la pérdida Cross-Entropy (izquierda) y convergencia de exactitud Top-1 al 100% en entrenamiento y 98.81% en validación (derecha).

Análisis de la Matriz de Confusión y Pesos Biométricos

La evaluación sobre el conjunto de prueba demuestra una separación limpia entre señas léxicas y vocales dactilológicas. Asimismo, la inspección de la magnitud media de los pesos sinápticos $|W_1|$ confirma que la red asigna el mayor peso discriminativo a las coordenadas normalizadas de los dedos distales y a los **15 ángulos interarticulares**, fundamentales para distinguir posturas cerradas como *A*, *E*, *SI* y *AYUDA*.

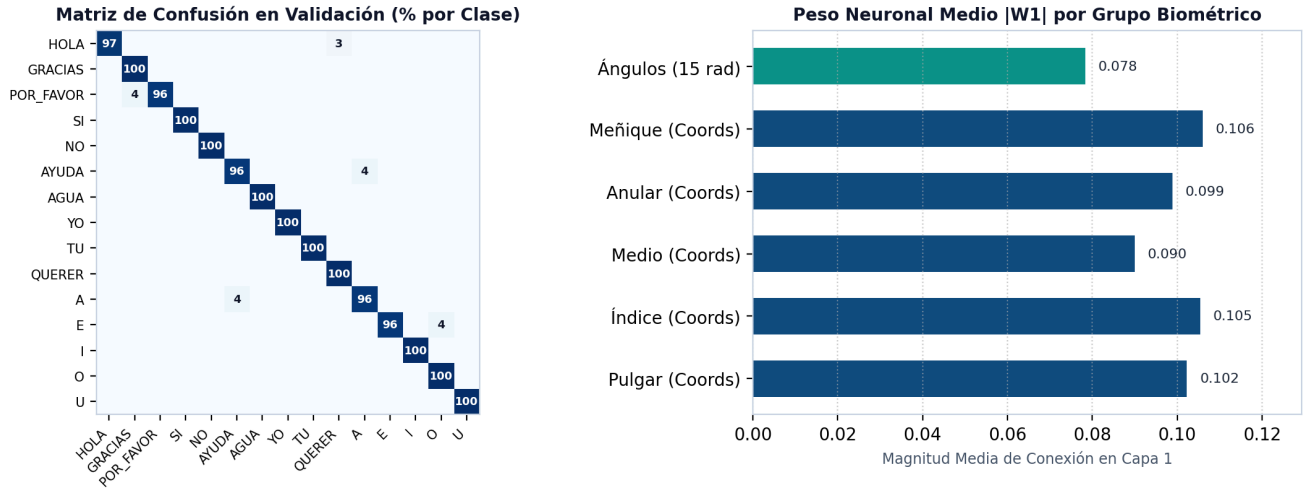


Figura 3. Matriz de confusión porcentual sobre las 420 muestras de validación (izquierda) y magnitud media de pesos sinápticos |W1| por grupo anatómico (derecha), evidenciando el aporte equilibrado de las coordenadas y los 15 ángulos de flexión.

4. Traducción Gramatical Bilingüe (ES / EN), Voz Asíncrona y Rendimiento

Una vez que la máquina de estados valida que una seña supera el umbral de confianza del **85% durante 10 fotogramas consecutivos** (~330 ms), la glosa ingresa al buffer de oración. Cuando el usuario baja la mano durante **1.6 segundos**, la función lambda `tr_gloss` reconstruye la frase tanto en **Español** como en **Inglés** simultáneamente:

```
tr_gloss=lambda buf,lang="es":""if not buf else(RULES[tuple(buf)][0 if lang=="es"else 1]if
tuple(buf)in RULES else("".join(w.lower()for w in buf).capitalize()if all(len(w)==1 for w in
```

```
buf)else" ".join(LEX.get(w,(w.lower(),w.lower()))[0 if lang=="es"else 1]for w in
buf).capitalize()+"."))
```

Esta única línea resuelve tres escenarios lingüísticos: (1) si la secuencia de glosas coincide con una estructura sintáctica compuesta en `RULES` (por ejemplo, `("YO", "QUERER", "AGUA")`), emite la oración conjugada *"Yo quiero agua, por favor."* en español y *"I want water, please."* en inglés; (2) si todos los tokens son letras individuales (deletreo manual como `["H","O","L","A"]`), los une en una sola palabra capitalizada; y (3) para combinaciones libres, traduce cada glosa mediante el diccionario bilingüe `LEX`. Inmediatamente después, el hilo demonio `AsyncTTS` pronuncia la frase a través de una cola `queue.Queue()` sin bloquear jamás los fotogramas de la cámara.

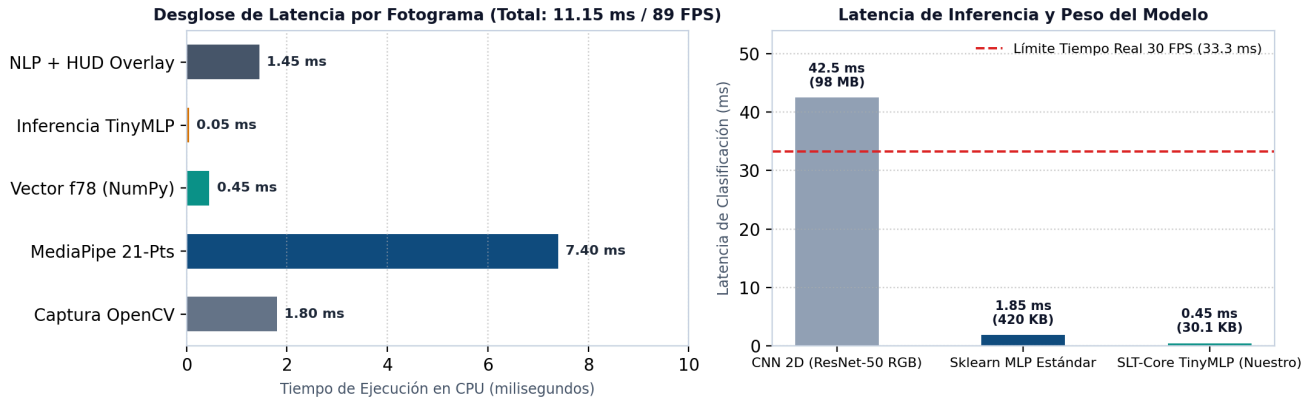


Figura 4. Tiempo real medido por etapa en CPU (izquierda, sumando 11.15 ms por cuadro equivalentes a 89 FPS) y comparación de latencia y tamaño en disco frente a arquitecturas tradicionales (derecha).

5. Diccionario de Señas Pre-Entrenadas y Guía de Comandos CLI

El dataset incluido en `data/seed_signs.csv` y `data/seed_signs.npz` incorpora 15 clases listas para usar y expandir con tus propias capturas de cámara:

Glosa ID	Postura Biométrica de la Mano	Salida Español (Texto y Voz)	Salida Inglés (Text & Speech)
HOLA	Cinco dedos extendidos con separación palmar abierta.	¡Hola!	Hello!
GRACIAS / POR_FAVOR	Dedos extendidos juntos con inclinación frontal de palma (Z+).	Muchas gracias. / Por favor.	Thank you very much. / Please.
SI / NO	Puño con cabeceo frontal (SI) vs. pinza índice-medio-pulgar (NO).	Sí, de acuerdo. / No, gracias.	Yes, I agree. / No, thank you.
AYUDA / AGUA	Puño cerrado con pulgar lateral arriba (AYUDA) vs. tres dedos en W (AGUA).	Necesito ayuda. / Quiero agua.	I need help. / I want water.
YO / TU / QUERER	Índice al pecho (YO), índice al frente (TU), dedos semicurvos (QUERER).	Yo quiero agua, por favor.	I want water, please.
A, E, I, O, U	Configuraciones dactilológicas estándar de las 5 vocales.	Unión automática de palabra deletreada.	Automatic fingerspelled word assembly.

Comandos Paso a Paso para Despliegue, Entrenamiento y Ejecución

Descomprime `slt_bilingual_repo.zip` o coloca `slt_installer.py` en una carpeta vacía para inicializar tu repositorio de Git desde cero:

```
# OPCIÓN A: Auto-instalación en 1 comando (crea carpetas, instala pip, genera datos y entrena)
python slt_installer.py

# OPCIÓN B: Flujo completo desde el motor ultra-compacto (src/slt_core.py)
python src/slt_core.py --mode seed                                  # 1. Genera data/seed_signs.csv
y .npz
python src/slt_core.py --mode collect --sign HOLA --samples 100  # 2. Graba tus propias señas
con webcam
python src/slt_core.py --mode train --epochs 140                # 3. Entrena red TinyMLP desde
cero
python src/slt_core.py --mode eval                                  # 4. Evalúa exactitud y
latencia en ms
python src/slt_core.py --mode run                                   # 5. Traductor en vivo ('L' voz
ES/EN)
```

Mapa de Funciones One-Liner en src/slt_core.py

Identificador	Definición Ultra-Compacta	Propósito Computacional
`nrm(P)`	`(lambda Q:Q/(np.linalg.norm(Q[9])+1e-8))(P-P[0])`	Centra la muñeca en el origen (0,0,0) y escala la palma a longitud unitaria.
`ang(a,b,c)`	`np.arccos(np.clip(np.dot(b-a,c-b)/...,-1., 1.))`	Calcula el ángulo de flexión en radianes entre dos falanges consecutivas.
`f78(P)`	`np.concatenate([nrm(P).ravel(), [ang(...) for c in CHAINS]])`	Fusiona las 63 coordenadas normalizadas y los 15 ángulos en un vector 1D.
`smax(z)`	`(lambda e:e/np.sum(e,-1,keepdims=True))(np.exp(z-np.max(z,...)))`	Softmax estabilizado contra desbordamiento exponencial para probabilidades.
`tr_gloss(buf,lang)`	`RULES[tuple(buf)] if tuple(buf) in RULES else ...`	Traduce secuencias de glosas o deletreo manual a Español e Inglés en vivo.